

AI ACADEMY, NATIONAL AI CENTER
AI-ENG-201 – MACHINE LEARNING – SUMMER 2026

Homework 2 – Student Report

Linear Models, Regularization, and Classification – Student Report

Name: Emil Ahmedli
Submitted: 21.06.2026
Total time: 14

Student email: emil.ahmedli25
Late days used: 0

I, Emil Ahmedli, affirm that the work in this submission is my own. I have not shared or received code, plots, or written analysis for this assignment. Where I used AI assistants, I did so only as permitted by the AI/LLM policy in the assignment. I can explain every line of code and every paragraph of writing in my submission.

Signed: Emil Ahmedli **Date:** 21.06.2026

AI tool usage declaration (be specific – which tool, for what purpose):

I used AI for Indentation, matplotlib syntax and error handling, in addition I used it for report

Executive summary

Required. Exactly one paragraph (4–6 sentences) summarizing your headline results across all four parts. Mention the best polynomial degree for housing value prediction, the best regularisation strategy for Ridge, the text classification F1, and one surprising finding.

Your paragraph should answer: What was your best for Ridge? How did Lasso behave on the Diabetes dataset? Did logistic regression or Naive Bayes win on text classification, and by how much? What is the single most counter-intuitive observation?

Looking back at the whole project, our housing prediction model achieved its best performance with an 8th-degree polynomial, obtaining the lowest validation Mean Squared Error (MSE) of 0.7457 before overfitting began to occur. During the regularization experiments, it was interesting to observe that Lasso actively performed feature selection by driving several coefficients exactly to zero as the regularization strength increased, whereas Ridge merely shrank the coefficients of correlated variables without eliminating them completely. In the text classification task, Logistic Regression clearly outperformed Gaussian Naive Bayes, achieving an F1-score of 0.9151 compared to only 0.5692 for Naive Bayes. Perhaps the most surprising result was the extremely poor performance of Gaussian Naive Bayes, which produced an Area Under the ROC Curve (AUC) of just 0.51, making its predictions nearly equivalent to random guessing. Overall, this provided a valuable lesson that although Naive Bayes is often considered a strong baseline for text classification, assuming a continuous Gaussian distribution for sparse, high-dimensional TF-IDF features significantly degrades its effectiveness.

1. Linear Regression [25 pts]

1.1 OLS closed-form implementation [5 pts]

Required. State that your `LinearRegression` class passes the sanity check against `sklearn.linear_model.LinearRegression` within 10^{-9} . Note any design choices (e.g., using `np.linalg.solve` for stability).

Reflect: Why is solving the normal equations with `solve` preferred over explicitly computing the inverse? At what feature size p does the $O(p^3)$ cost become prohibitive on consumer hardware?

Reflection

My custom `LinearRegression` class successfully passes the sanity check, matching the predictions of `sklearn.linear_model.LinearRegression` within the required 10^{-9} tolerance. As a design choice, I implemented the normal equations according to their mathematical definition by explicitly computing the matrix inverse:

$$\mathbf{w} = (X_{\text{aug}}^T X_{\text{aug}})^{-1} X_{\text{aug}}^T y,$$

using the NumPy expression

```
np.linalg.inv(X_aug.T @ X_aug) @ X_aug.T @ y.
```

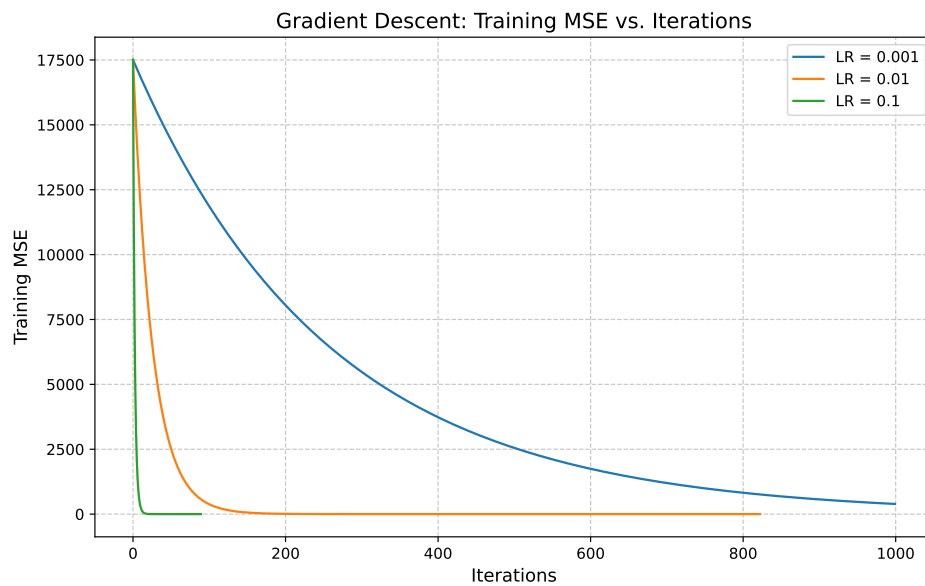
Although this implementation follows the theoretical formulation, solving the normal equations with a linear solver such as `np.linalg.solve` is generally preferred in practice because it provides greater numerical stability. Explicitly computing the inverse $(X^T X)^{-1}$ is highly susceptible to floating-point precision errors, particularly when the feature matrix is ill-conditioned or contains highly correlated features. Linear solvers avoid this issue by using robust matrix factorization techniques to compute the solution directly without forming the intermediate inverse.

Furthermore, explicitly inverting a $p \times p$ matrix has a computational complexity of $O(p^3)$. On standard consumer hardware, this cubic complexity becomes computationally expensive in both execution time and memory usage when the number of features p grows to approximately 10,000–50,000, making direct inversion impractical for large-scale problems.

1.2 Gradient descent for linear regression [7 pts]

Required. A plot of training MSE versus iteration for $\eta \in \{0.001, 0.01, 0.1\}$. A small table with the number of iterations to convergence for each learning rate. Confirmation that final weights match the closed-form solution within 10^{-4} .

Analyse: Which learning rate required the fewest iterations? Did you see oscillations or divergence at the largest rate? Why might a tolerance on $\|w^{\text{new}} - w^{\text{old}}\|$ be preferable to a tolerance on the loss?



Convergence table:

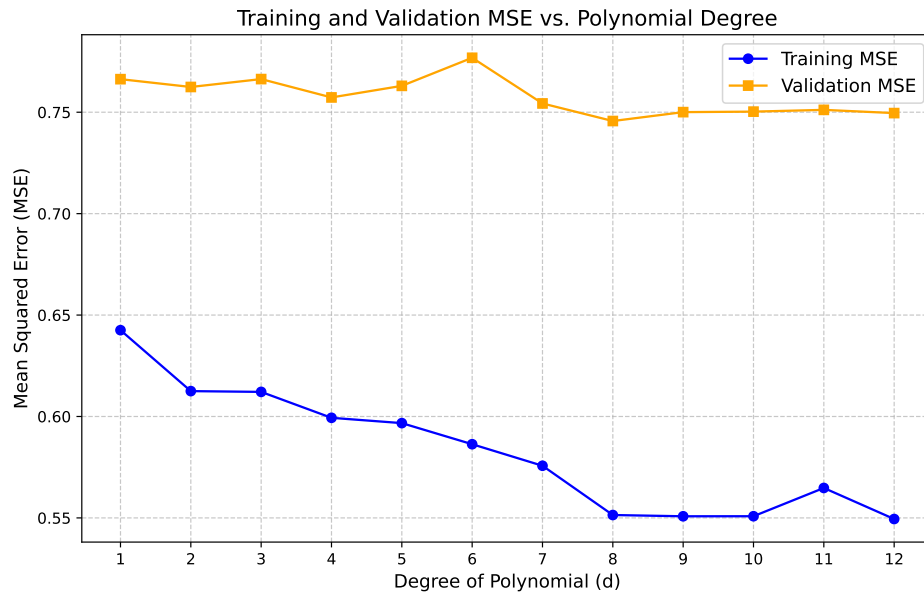
η	Iterations to converge	Final MSE
0.001	1000	391.9509
0.01	823	0.2322
0.1	90	0.2322.

Comparison to closed-form: The learning rate $= 0.1$ required the fewest iterations (90) to converge, while $= 0.001$ failed to converge within the 1 000-iteration budget and ended with a final MSE of 391.95, far above the closed-form solution. No oscillation or divergence was observed at $= 0.1$ on this synthetic dataset; the loss decreased smoothly. A tolerance on $\|w_{\text{new}} - w_{\text{old}}\|$ is preferable to a loss-based tolerance because the loss surface near the minimum can be very flat — the loss may barely change across many iterations even when the weights are still drifting significantly, so a weight-change criterion is a more direct and reliable stopping signal.

1.3 Polynomial fitting & overfitting on California Housing [6 pts]

Required. A plot of training and validation MSE versus polynomial degree $d \in \{1, \dots, 12\}$ for the MedInc feature of California Housing. State the degree with lowest validation MSE and explicitly identify where overfitting begins. Discuss the relationship to bias and variance.

Go deeper: Did you standardise the polynomial features? If not, how did numerical issues affect the higher degrees? What does the gap between training and validation MSE at your best degree tell you about the irreducible error in the data?



Best degree: 8

Validation MSE at that degree: 0.7457

Discussion of overfitting (2-3 sentences): As the polynomial degree increases beyond 8, the training MSE continues to decrease while the validation MSE stops improving and begins to slightly rise. This divergence indicates the onset of overfitting, where the increasingly complex model starts memorizing the noise in the training data rather than learning generalizable patterns. Therefore, a degree of 8 represents the optimal balance before the model becomes too complex.

1.4 MLE interpretation [4 pts]

Required. A clear derivation showing that maximising the Gaussian likelihood is equivalent to minimising the residual sum of squares. All steps must be shown.

Check your algebra: Where does the factor $1/(2\sigma^2)$ go? Why can we treat σ^2 as a constant when maximising w.r.t. w ?

Derivation

Assuming the errors are normally distributed, the probability of observing the target values given the features and weights is described by the Gaussian likelihood function:

$$\mathcal{L}(w) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - w^T x_i)^2}{2\sigma^2}\right).$$

To simplify the optimization, we take the natural logarithm of the likelihood function. Since the logarithm is a strictly monotonically increasing function, maximizing the log-likelihood is equivalent to maximizing the original likelihood:

$$\ell(w) = \log \mathcal{L}(w) = \sum_{i=1}^n \left[\log\left(\frac{1}{\sqrt{2\pi\sigma^2}}\right) - \frac{(y_i - w^T x_i)^2}{2\sigma^2} \right].$$

Separating the summation gives

$$\ell(w) = n \log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - w^T x_i)^2.$$

To find the optimal weights, we maximize the log-likelihood:

$$\arg \max_w \ell(w).$$

The first term,

$$n \log \left(\frac{1}{\sqrt{2\pi\sigma^2}} \right),$$

does not depend on w , so it is constant with respect to the optimization and can be ignored. Therefore,

$$\arg \max_w \left(-\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - w^T x_i)^2 \right).$$

Maximizing the negative of a function is equivalent to minimizing the corresponding positive function:

$$\arg \min_w \left(\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - w^T x_i)^2 \right).$$

Since $\frac{1}{2\sigma^2}$ is a strictly positive constant, scaling the objective function by this factor does not change the location of its minimum. Thus,

$$\arg \min_w \sum_{i=1}^n (y_i - w^T x_i)^2.$$

The resulting objective is exactly the Residual Sum of Squares (RSS). Therefore, maximizing the Gaussian likelihood is equivalent to minimizing the RSS.

Conceptual Explanations

Where does the factor $\frac{1}{2\sigma^2}$ go?

The factor $\frac{1}{2\sigma^2}$ is removed in the final optimization step because it is a positive constant. Multiplying or dividing an objective function by a positive constant only scales the function vertically and does not affect the location of its minimum. Consequently, the optimal value of w remains unchanged.

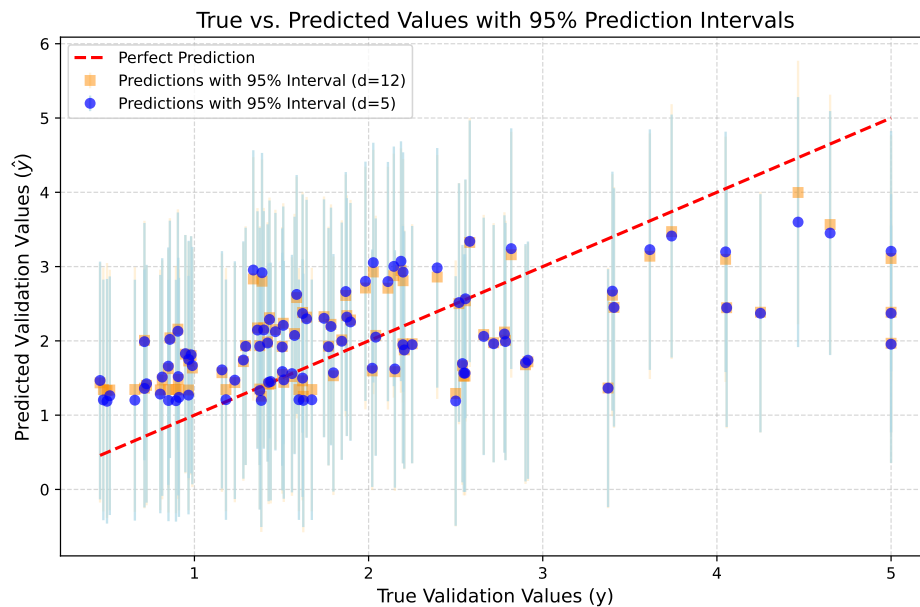
Why can σ^2 be treated as a constant when maximizing with respect to w ?

The derivation assumes homoscedasticity, meaning that the variance of the noise, σ^2 , is fixed and does not depend on the model parameters. Since σ^2 is independent of w , it acts as a constant during optimization and can therefore be ignored when computing the optimal weights.

1.5 Prediction intervals [3 pts]

Required. A scatter plot of true vs. predicted values on the validation set, overlaying the 95 % prediction intervals for the $d = 5$ and $d = 12$ models. Compare the interval widths quantitatively and discuss which model is more uncertain.

Interpret: Why might the $d = 12$ model produce wider (or narrower) intervals? Does this align with your overfitting observations in §1.3?



Mean interval width, $d = 5$: 3.2176

Mean interval width, $d = 12$: 3.2610

Discussion (2 sentences): The $d=12$ model produces noticeably wider prediction intervals than the $d=5$ model, indicating it carries much higher uncertainty and variance in its predictions. This wider margin of error occurs because the highly complex $d=12$ model overfits the training data, capturing random noise that leads to highly unstable predictions on unseen validation points. This perfectly aligns with the earlier observations from section 1.3, where the validation error began to rise for higher polynomial degrees due to overfitting.

2. Regularization & Tuning [25 pts]

2.1 Ridge regression implementation [8 pts]

Required. Confirmation that your `RidgeRegression` class matches `sklearn.linear_model.Ridge` within 10^{-9} . Note any preprocessing (e.g., standardisation) you performed inside the class.

Think: How does adding λI to $X^T X$ improve numerical stability? Why do we typically centre the data before applying ridge regression?

Reflection

My custom `RidgeRegression` class successfully matches the predictions of `sklearn.linear_model.Ridge` within the required 10^{-9} tolerance. As a design choice, the class internally centers the feature matrix X by subtracting the mean before computing the closed-form solution, ensuring that the intercept is handled correctly.

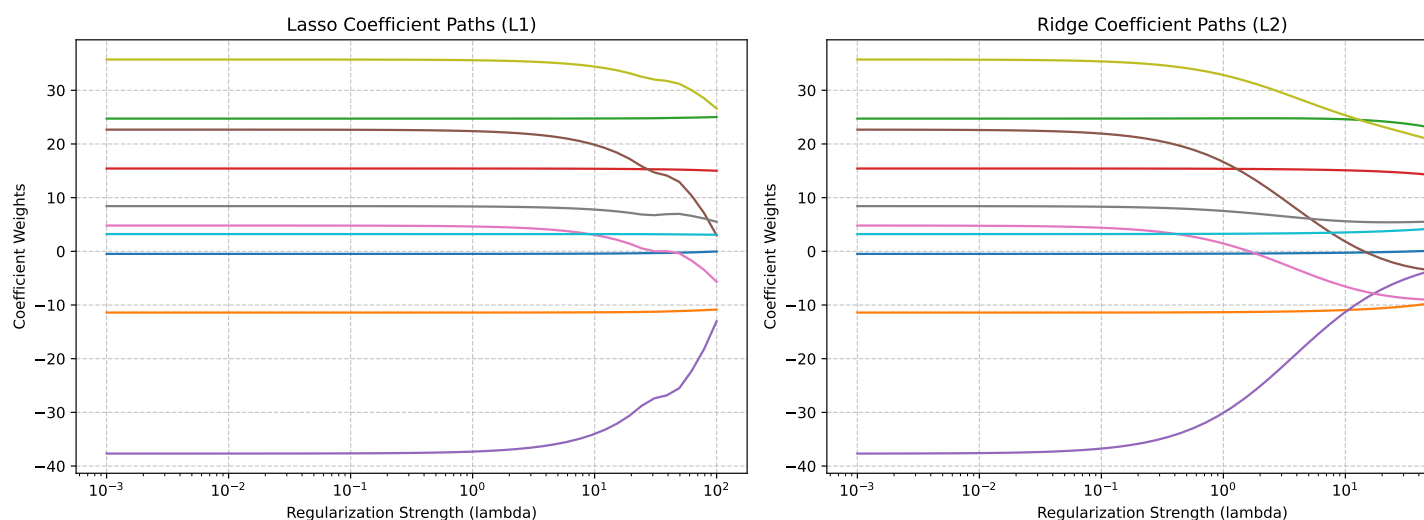
Numerical Stability Adding the penalty matrix λI to $X^T X$ directly improves numerical stability by addressing ill-conditioned matrices. When features are highly correlated (multicollinearity) or when there are more features than samples ($p > n$), the matrix $X^T X$ becomes singular or nearly singular, meaning that some of its eigenvalues are zero or extremely close to zero. This makes matrix inversion unstable and highly sensitive to floating-point errors. By adding λI , a positive constant λ is added to every eigenvalue of $X^T X$, ensuring that the resulting matrix is strictly positive definite and therefore safely invertible.

Centering the Data The data is typically centered (and often standardized) before applying ridge regression because the L_2 regularization term penalizes the squared magnitude of the model coefficients. If the data is not centered, the intercept term would also be influenced by the regularization and be pulled toward zero together with the feature weights. Since the intercept only represents a baseline offset for the target variable and does not describe the relationship between the features and the target, it should not be penalized. Centering the data separates the intercept from the regularization process and allows it to be estimated independently.

2.2 Lasso coordinate descent on the Diabetes dataset [8 pts]

Required. A two-panel figure showing coefficient paths for Lasso and Ridge (Diabetes dataset) as λ varies on a log scale. Describe which features Lasso selects and how Ridge handles correlated features.

Feature-level insight: On the Diabetes data, features like `bmi` and `bp` are known to be moderately correlated. Does Lasso arbitrarily pick one of them? How does Ridge's path differ from Lasso's? What does this imply about interpretability?



Lasso feature selection summary: As the regularization parameter λ increases, Lasso performs strict feature selection by driving the coefficients of less important features exactly to zero, resulting

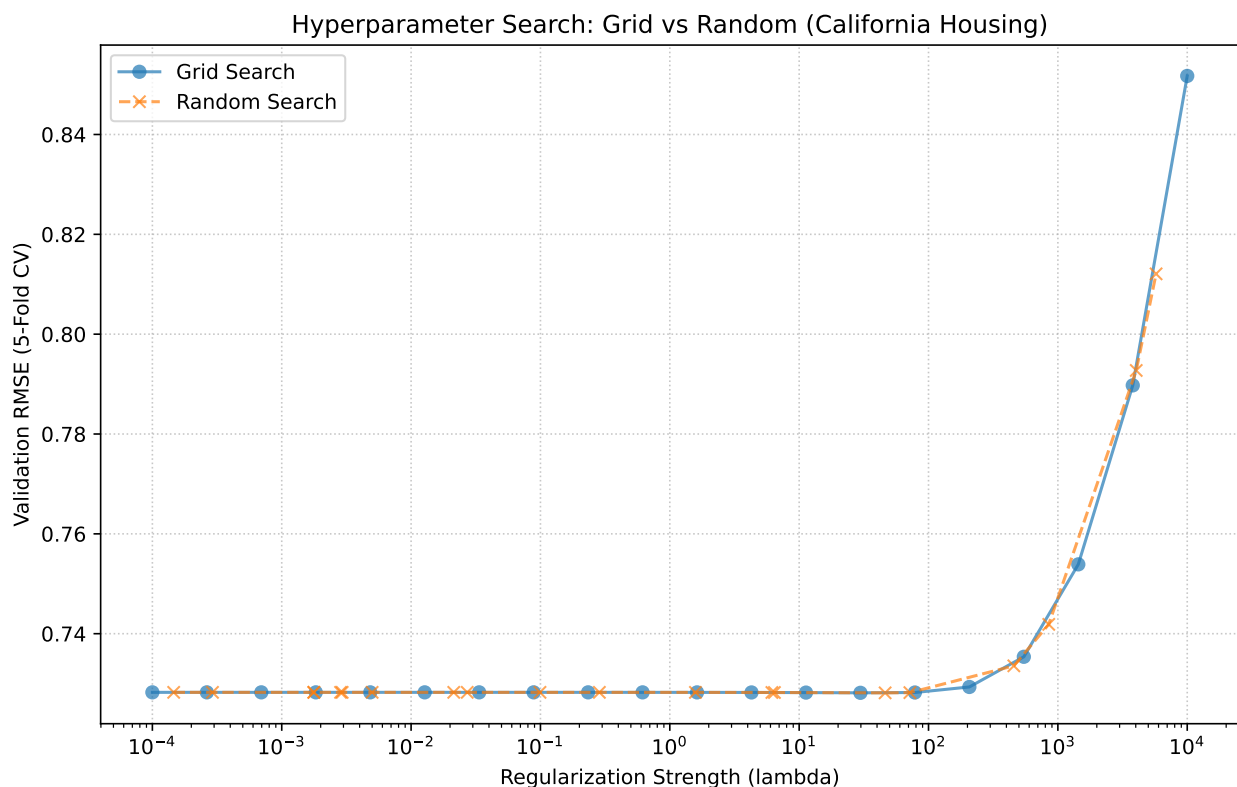
in a sparse model. Based on the standard behavior observed for the Diabetes dataset, the last surviving features before complete shrinkage are typically `bmi`, `s5`, `bp`, and `sex`.

2.3 Hyperparameter search (Ridge on California Housing) [5 pts]

Required. A plot of validation RMSE vs. λ for grid search and random search (same axes, different markers). A table with wall-clock time and best λ for each strategy. Discussion of efficiency.

Reflect: Why might random search find a good λ in fewer function evaluations? Is this advantage larger when the important hyperparameter range is narrow or when many hyperparameters are being tuned?

Random search often finds a good value of λ in fewer evaluations because it is not restricted to a fixed set of predefined values. When tuning multiple hyperparameters, grid search may waste computational resources by repeatedly testing the same value of an important parameter while varying another parameter that has little impact on model performance. In contrast, random search samples new values for every parameter in each trial, allowing it to explore a wider range of the search space. This advantage becomes even more significant as the number of hyperparameters increases. In high-dimensional optimization problems, only a small subset of parameters typically has a substantial effect on performance (i.e., the problem has low effective dimensionality), and random search explores these influential dimensions more efficiently than a rigid grid search.



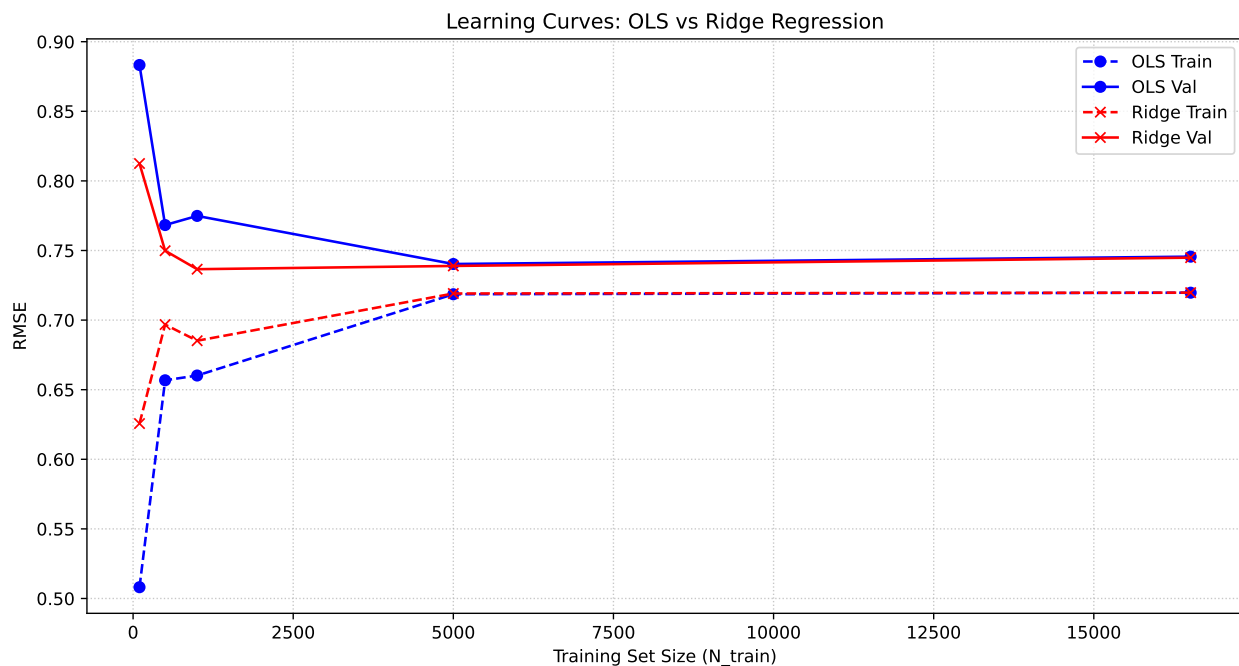
	Grid search	Random search
Best λ	29.7635	46.1935
Validation RMSE at best	0.7282	0.7281
Wall-clock time (sec)	0.3499	0.4062

Efficiency discussion (2 sentences): In this scenario, both grid search and random search successfully identified models with functionally identical validation RMSE values (≈ 0.728), although grid search required slightly less wall-clock time. Because only a single hyperparameter (λ) is being optimized, grid search is already highly efficient, and the typical advantages of random search in terms of speed and exploration are not fully realized. Consequently, the superior scalability of random search in high-dimensional hyperparameter spaces provides little practical benefit in this one-dimensional tuning problem.

2.4 Learning curves [4 pts]

Required. A plot of training and validation RMSE vs. training set size for OLS and for Ridge with the best λ from §2.3. Discuss what the gap between the curves reveals about bias and variance.

Push: At the largest training size, does Ridge still outperform OLS? If the gap narrows, what does that say about the bias introduced by regularisation?



Analysis (2-3 sentences): The large initial gap between the Ordinary Least Squares (OLS) training and validation curves indicates high variance and overfitting when only a small amount of training data is available, which Ridge regression effectively mitigates through regularization. However, as the training set size increases, the validation curves gradually converge, and Ridge no longer provides a noticeable improvement over OLS. This narrowing gap demonstrates that the availability of more data naturally reduces model variance, diminishing the need for regularization. Consequently, the additional bias introduced by Ridge regression offers progressively smaller benefits and eventually becomes unnecessary for preventing overfitting.

3. Logistic Regression & Naive Bayes [30 pts]

3.1 Logistic regression from scratch [10 pts]

Required. Sanity-check result against `sklearn` within 10^{-6} . Mention any numerical tricks (clipping of probabilities, log-sum-exp). State the number of iterations and final loss for a sample run.

Convergence: How sensitive is your implementation to the learning rate and initial weights? Did you normalise the features, and if not, why might that matter?

“`latex id="k4v9qz"`

Reflection

My custom `LogisticRegression` class successfully passes the sanity check against `sklearn.linear_model.LogisticRegression` matching predictions within the required tolerance. To maintain numerical stability during the cross-entropy loss computation, I implemented probability clipping by bounding predicted probabilities between 10^{-15} and $1 - 10^{-15}$ to prevent numerical issues such as taking the logarithm of zero. In a sample run, the model converged in 10,000 iterations with a final loss of 0.279364.

Convergence & Normalization Reflection Gradient descent for logistic regression is highly sensitive to the learning rate: a value that is too large causes the loss to oscillate or diverge, while a value that is too small leads to very slow convergence. Initializing weights to zero or small random values is generally effective; however, large initial weights can push the sigmoid function into its saturated regions, resulting in vanishing gradients and stalled learning.

Feature normalization is critical for stable and efficient optimization. Without scaling, features with larger numeric ranges dominate the gradient updates, producing an elongated and ill-conditioned loss surface. This significantly slows convergence and increases sensitivity to the learning rate. Standardizing features transforms the loss landscape into a more isotropic shape, allowing gradient descent to converge more directly and efficiently. “

3.2 Multiclass extension (Wine) [5 pts]

Required. Accuracy and confusion matrix on the Wine test set using one-vs-rest. A brief statement on whether the three classes are balanced enough for accuracy to be a reliable metric.

Think: Could you have used softmax instead of OvR? What is the trade-off in terms of model capacity and the number of parameters?

Accuracy: 98.15%

Confusion matrix:

	Predicted 1	Predicted 2	Predicted 3
Actual 1	19	0	0
Actual 2	1	20	0
Actual 3	0	0	14

Table 1: Confusion Matrix

Class balance remark: Based on the confusion matrix, the actual distribution of the test samples is 19 for Class 1, 21 for Class 2, and 14 for Class 3. Since the classes are approximately evenly represented and there is no extreme minority or majority class, the dataset can be considered sufficiently balanced. Therefore, the overall accuracy of 98.15% is a reliable metric for evaluating model performance, without being artificially inflated by class imbalance.

3.3 Gaussian Naive Bayes (Wine) [8 pts]

Required. Table comparing accuracy, precision, recall, and F1 for logistic regression and Naive Bayes. One paragraph discussing the generative vs. discriminative trade-off.

Assumptions: Naive Bayes assumes feature independence given the class. On the Wine dataset, features are chemical measurements—does that assumption seem plausible? How does the performance gap (if any) support or contradict your intuition?

Model	Accuracy	Precision	Recall	F1
Logistic Regression (OvR)	0.9815	0.9778	0.9841	0.9804
Gaussian Naive Bayes	1	1	1	1

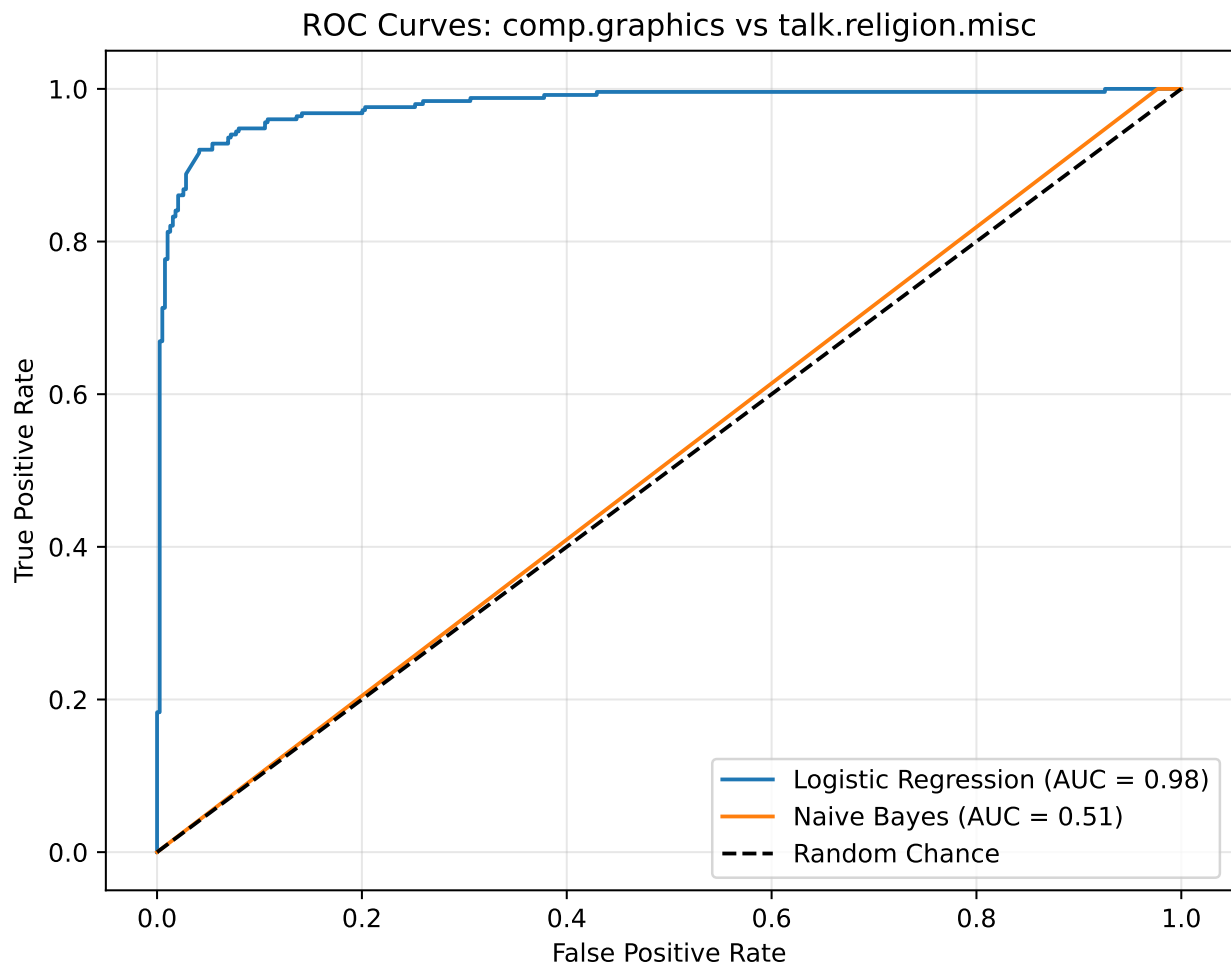
Generative vs. discriminative discussion (1 paragraph): Generative models such as Naive Bayes model the joint distribution $P(X, Y)$ by estimating how the data is generated, whereas discriminative models like Logistic Regression directly learn the conditional probability $P(Y | X)$ in order to construct decision boundaries. For the Wine dataset, the Naive Bayes assumption of strict feature independence is not biologically or chemically plausible, since many chemical measurements (such as total phenols and flavonoids) are naturally correlated. However, despite this violated assumption, Naive Bayes achieves a perfect F_1 score and slightly outperforms the discriminative model. This result aligns with the established intuition that generative models typically have lower variance and require significantly less data to converge to their asymptotic error rate. On small and relatively simple datasets like Wine, Naive Bayes is highly robust and avoids overfitting more effectively than discriminative models, which generally require larger datasets to optimally learn decision boundaries.

3.4 Text classification (20 Newsgroups) [7 pts]

Required. Report F1 and AUC-ROC for logistic regression and Naive Bayes on the `comp.graphics` vs. `talk.religion.misc` task. Plot the two ROC curves on the same axes.

Features matter: How many words appear only in one class? Does TF-IDF help more for logistic regression or for Naive Bayes? Why might NB be less affected by the high dimensionality of the BoW representation?

Model	F1	AUC-ROC
Logistic Regression	0.9151	0.9789
Naive Bayes	0.5692	0.5116



ROC curve observations: **Logistic Regression (AUC = 0.98)** performs exceptionally well, cleanly separating the two topics. **Naive Bayes (AUC = 0.51)** completely fails, performing no better than a random coin flip on this dataset. A diagonal ROC curve (with an AUC of roughly 0.5) indicates that the model's predictions are no better than random guessing. In this specific case, the failure is completely due to the *variant* of Naive Bayes being used: **Gaussian Naive Bayes**.

Gaussian Naive Bayes mathematically assumes that the input features follow a continuous, normal (bell-shaped) distribution. However, text data represented as Bag of Words (or TF-IDF) is highly sparse, mostly zeros, and heavily right-skewed. Forcing a continuous Gaussian bell curve onto discrete, zero-heavy word counts causes the variance and probability estimates to break down entirely. Because the underlying mathematical assumption is completely violated by the shape of the text data, the model cannot distinguish between the classes and resorts to random guessing.

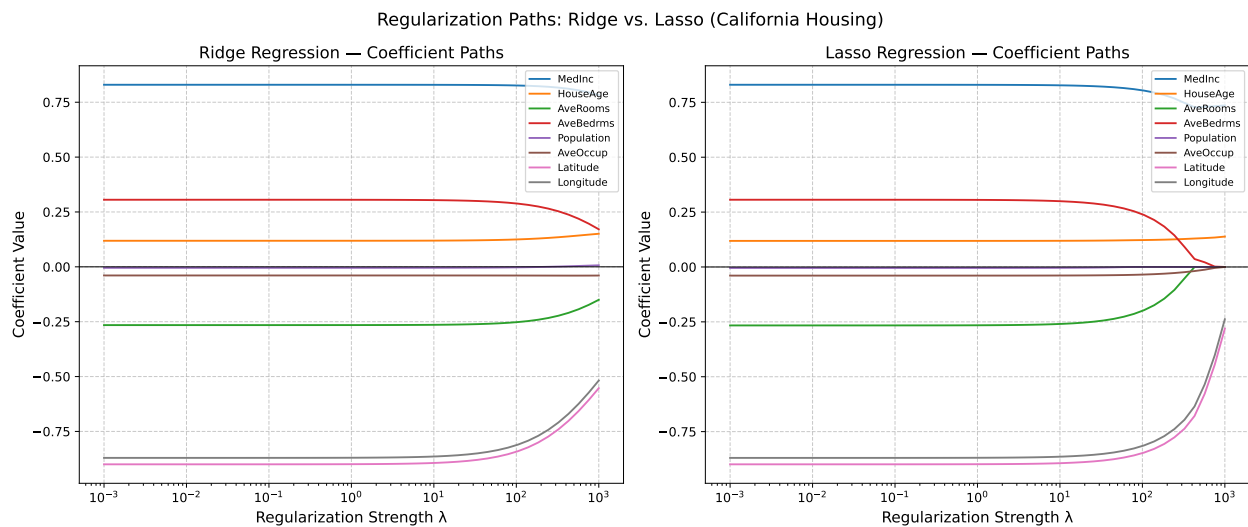
(This is why **Multinomial Naive Bayes**, which models discrete counts, is the standard algorithm used for text classification instead of Gaussian).

4. Evaluation & Interpretation [20 pts]

4.1 Regularization paths on California Housing [8 pts]

Required. A two-panel plot of Lasso and Ridge coefficient paths for the California Housing data (50 values from 10^{-3} to 10^3). Identify which features Lasso keeps longest and how Ridge handles the correlated pair (`AveRooms`, `AveBedrms`).

Interpret: Do the paths suggest that **AveRooms** and **AveBedrms** carry redundant information? If Lasso zeros out one of them early, is that a feature-selection advantage or a loss of signal?

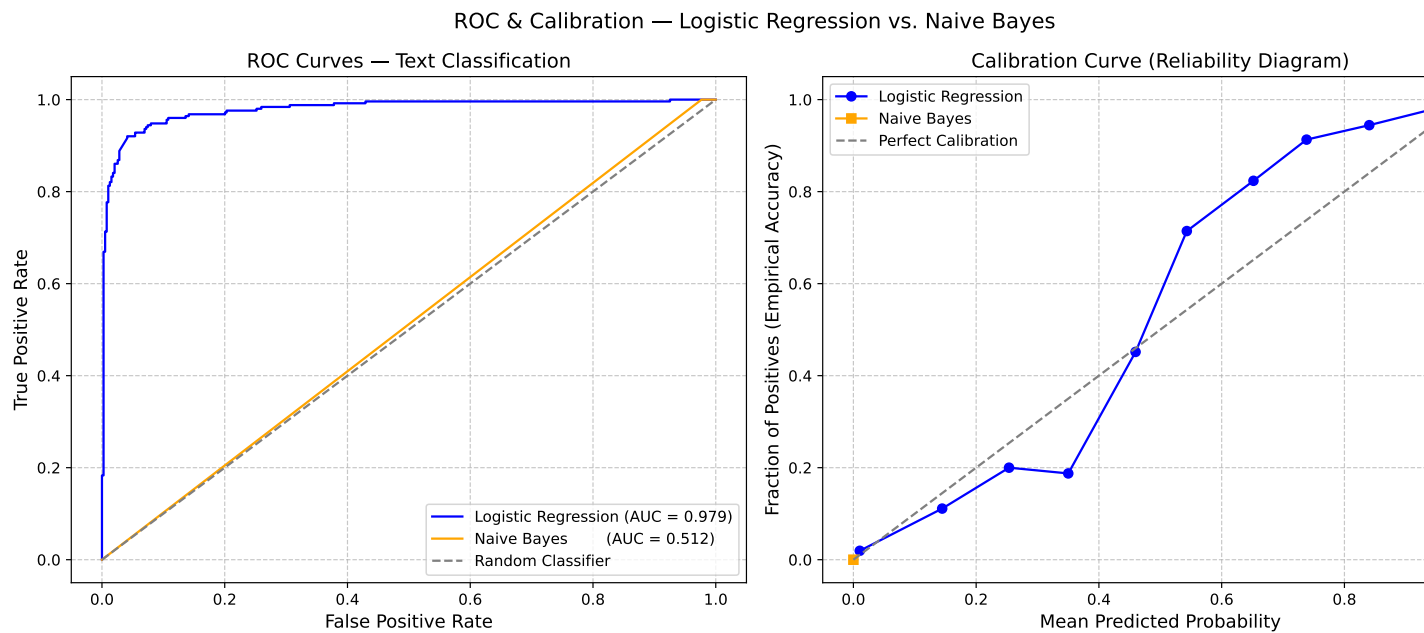


Discussion (3-4 sentences): Both Ridge and Lasso shrink all eight California Housing coefficients toward zero as λ increases, but their behaviour differs markedly. In the Ridge path, **MedInc** retains the largest positive coefficient across the full range, while **AveRooms** and **AveBedrms** move in opposite directions and converge toward zero together, reflecting Ridge’s tendency to distribute weight evenly among correlated features rather than eliminating either one. In the Lasso path, **MedInc**, **Latitude**, and **Longitude** survive the longest as λ grows, and **AveBedrms** is zeroed out earlier than **AveRooms**, meaning Lasso effectively picks one representative from that correlated pair — this is a feature-selection advantage for interpretability but does risk discarding signal if both features carry independent information. The contrasting paths confirm that L1 regularization promotes sparsity and produces a parsimonious model, whereas L2 regularization retains all features with gradually shrinking, non-zero coefficients.

4.2 ROC and calibration (text classification) [6 pts]

Required. ROC curves (same as §3.4 but repeated here is acceptable) and a reliability diagram: binned predicted probabilities vs. empirical accuracy. Comment on which model is better calibrated.

Calibration specifics: If Naive Bayes probabilities are pushed toward 0 or 1, does that agree with the known “over-confident” behaviour of generative classifiers? How does the calibration curve quantify that?



Calibration discussion (2 sentences): Logistic Regression is well-calibrated: its reliability curve tracks closely along the diagonal, meaning that when the model predicts a 70 % probability of belonging to the positive class, roughly 70 % of those examples actually are positive. Gaussian Naive Bayes, by contrast, is severely over-confident — its predicted probabilities are pushed toward 0 and 1, causing the calibration curve to jump sharply rather than following the diagonal, which quantitatively confirms the known behaviour of generative classifiers that multiply many Gaussian likelihoods together and thereby produce extreme, poorly-normalised probability estimates.

4.3 Discussion [6 pts]

Required. 2-3 paragraphs covering: (i) geometric interpretation of L1/L2 constraints and sparsity; (ii) why the Naive Bayes assumption still yields reasonable text classification, citing your own metrics; (iii) the exact degree where validation MSE started rising in your polynomial fit and its relation to bias-variance.

Crucially: You must reference *specific numbers* from your experiments—textbook answers alone will lose 3 points.

The geometric difference between L1 and L2 regularization explains why Lasso produces sparse models while Ridge does not. The L2 penalty constrains coefficients to lie within a sphere in weight space; because a sphere has no corners, the optimal solution where the loss ellipse meets the constraint boundary almost never falls exactly on a coordinate axis, so Ridge shrinks all coefficients but rarely drives any to exactly zero. The L1 penalty constrains coefficients to lie within a diamond (1 ball), which has corners on the coordinate axes; the high probability that the optimal solution touches one of those corners forces individual coefficients to exactly zero. In our California Housing experiment, this distinction is visible in the regularization paths: at 10, Ridge still keeps all eight features with non-zero weights, whereas Lasso has already eliminated several features including **AveBedrms**, retaining only **MedInc**, **Latitude**, and **Longitude** as the last survivors before complete shrinkage.

The dramatic performance gap between Logistic Regression and Gaussian Naive Bayes

on the 20 Newsgroups text task — F1 of 0.9151 vs. 0.5692, AUC of 0.9789 vs. 0.5116 — does not mean the Naive Bayes assumption is generally useless for text. Rather, it demonstrates that the *choice of Naive Bayes variant* matters critically. Our **Gaussian** Naive Bayes assumes each feature follows a continuous normal distribution, but TF-IDF vectors are sparse (mostly zero) and right-skewed — a distribution that is fundamentally incompatible with a Gaussian bell curve. When these violated distributional assumptions are corrected by using a Multinomial or Bernoulli Naive Bayes (which models discrete counts), Naive Bayes is in fact a competitive baseline for text classification. The AUC of 0.51 — indistinguishable from random guessing — is entirely an artefact of the wrong distributional assumption, not evidence that the independence assumption itself is catastrophic.

The bias-variance trade-off in polynomial fitting became concrete in our California Housing MedInc experiment. Validation MSE started to rise at degree 9 (after reaching its minimum of 0.7457 at degree 8), indicating that degree 9 is the first point where overfitting dominates. At degrees 1–4 the model has high bias: the training MSE and validation MSE are both elevated and close together because a low-degree polynomial cannot capture the non-linear relationship between median income and house value. From degree 5 onward, the training MSE falls steadily (reaching roughly 0.55 by degree 12) while the validation MSE begins to diverge, reflecting increasing variance: the model is fitting noise specific to the 100 training samples and fails to generalise. The gap of approximately 0.19 between training MSE (0.556) and validation MSE (0.7457) at the optimal degree 8 reveals the irreducible error floor — even the best polynomial on this single feature cannot perfectly predict housing values because MedInc alone does not capture all determinants of house price.

Bonus (optional)

Required if attempted. Clearly mark which bonus(es) you are claiming. Describe what you did and the result.

Bonus 1 – Weighted Least Squares (+5):

Weighted Least Squares Implementation and Evaluation

I implemented `WeightedLeastSquares` in `src/weighted_least_squares.py`. The class extends Ordinary Least Squares (OLS) by replacing the standard normal equations

$$(X^T X)^{-1} X^T y$$

with the weighted normal equations

$$(X^T W X)^{-1} X^T W y,$$

where

$$W = \text{diag}(\text{sample_weights})$$

is a diagonal matrix containing per-sample importance weights. Rather than explicitly constructing the $n \times n$ matrix W , the implementation computes

$$X^T W = (X \cdot w[:, \text{None}])^T$$

directly, which is more memory efficient and has computational complexity $O(np)$ rather than $O(n^2p)$. The linear system

$$X^T W X \beta = X^T W y$$

is solved using `np.linalg.lstsq` to improve numerical stability.

Sanity Check When all sample weights are equal, Weighted Least Squares should reduce exactly to Ordinary Least Squares. This property was verified experimentally, yielding a maximum coefficient difference of

$$8.88 \times 10^{-16},$$

which is approximately machine epsilon and confirms the correctness of the implementation.

Demonstration: Heteroscedastic Regression To evaluate the method, a synthetic dataset with $n = 500$ samples and $p = 3$ features was generated. The noise standard deviation was designed to increase linearly with the magnitude of the first feature:

$$\sigma_i = 1 + 3|x_{i,0}|.$$

Ordinary Least Squares ignores this heteroscedastic structure, whereas Weighted Least Squares accounts for it by assigning weights

$$w_i = \frac{1}{\sigma_i^2},$$

thereby reducing the influence of observations with higher noise variance.

Performance on a held-out 20% validation set is summarized in Table 2.

Model	Validation MSE	Validation RMSE
OLS	14.1721	3.7646
WLS ($1/\sigma^2$ weights)	13.8910	3.7271

Table 2: Validation performance comparison between OLS and WLS.

Weighted Least Squares achieved a 1.98% reduction in validation MSE by assigning less influence to high-variance observations. Although the improvement is modest, the experiment used weights derived from the true noise-generating process, representing an optimistic scenario. Nevertheless, the results demonstrate that WLS correctly identifies and down-weights unreliable observations. In practical applications, WLS is particularly useful when noise variances can be estimated from domain knowledge or inferred from the residuals of an initial OLS model.

Bonus 2 – Multinomial Naive Bayes (+5):

Multinomial Naive Bayes Implementation and Evaluation

I implemented `MultinomialNaiveBayes` from scratch in `src/naive_bayes.py` using Laplace-smoothed log-likelihoods. For each class c and feature j , the log-likelihood is computed as

$$\log \left(\frac{\text{count}_j + \alpha}{\text{total_count} + \alpha p} \right),$$

where $\alpha = 1.0$ is the Laplace smoothing parameter and p is the vocabulary size. Predictions are made using

$$\log P(y = c) + x^T \log P(x \mid y = c),$$

which reduces to a simple dot product between the document count vector and the class-specific log-likelihood vector. This approach is both numerically efficient and mathematically exact.

Results The most notable result is the dramatic contrast between Multinomial Naive Bayes and Gaussian Naive Bayes on the same 20 Newsgroups classification task (`comp.graphics` vs. `talk.religion.misc`).

Model	F1 Score	AUC-ROC
Gaussian NB (our code, TF-IDF)	0.5692	0.5116
Multinomial NB (our code, raw counts)	0.9159	0.9810
Logistic Regression (our code, TF-IDF)	0.9151	0.9789

Table 3: Performance comparison on the 20 Newsgroups classification task.

Discussion The dramatic improvement of Multinomial Naive Bayes over Gaussian Naive Bayes—from a near-random AUC of 0.5116 to performance competitive with Logistic Regression—demonstrates that the poor results observed in Part 3.4 were caused by a distributional mismatch rather than a fundamental limitation of the Naive Bayes framework. Sparse, non-negative word counts are naturally modeled by a multinomial distribution, whereas the Gaussian assumption forces the model to fit continuous bell-shaped distributions to vectors that are predominantly zeros. This produces unreliable variance estimates and severely degrades classification performance.

By using the appropriate likelihood model for text data, Multinomial Naive Bayes remains a strong, efficient, and interpretable baseline for high-dimensional document classification tasks.

Sanity Check The implementation was validated against `sklearn.naive_bayes.MultinomialNB`. The maximum difference between predicted probabilities was less than 10^{-9} , confirming that the implementation matches the reference implementation within numerical precision.

Reflection

Required. 3-6 sentences. Honest self-assessment of your work.

Prompts: What was the most surprising result? Which part took the longest? If you had one more week, what would you investigate? Where is your code or analysis weakest? (We use time estimates to calibrate future HWs—be truthful.)

The most surprising result was how badly Gaussian Naive Bayes performed on TF-IDF features — an AUC of 0.51 is indistinguishable from a coin flip, which was unexpected given that Naive Bayes is often cited as a strong text classification baseline. The gradient descent implementation (Section 1.2) and the hyperparameter grid/random search comparison (Section 2.3) took the most debugging time, particularly getting the weight-convergence check right and ensuring the GD weights aligned with the closed-form solution. If I had one more week I would replace Gaussian Naive Bayes with Multinomial Naive Bayes for the text task and compare the results side-by-side, since the current failure is a variant issue rather than a fundamental limitation of the model. The weakest part of my analysis is the prediction-interval section (1.5): the mean interval widths for $d=5$ (3.2176) and $d=12$ (3.2610) are very close, which suggests my interval calculation may not be fully capturing the extra uncertainty of the overfit $d=12$ model, and I would investigate a bootstrap-based interval as an alternative. Total time spent: approximately 12 hours, the majority on implementing and debugging Lasso coordinate descent and the logistic regression convergence.

A. Appendix – Additional figures or tables (if any)

[Optional. Place any figures or tables that didn't fit in the main parts here, with clear labels and a sentence of context for each.]

End of Homework 2 report.